

SE3308 Algorithm Design I — Assignment 1

Exercises 0.1, 0.2, 1.14, 1.20, 1.31, 1.35

Xilang Chen 524031910668

Throughout, $\log$ denotes $\log_2$ and $\ln$ the natural logarithm.

Exercise 0.1

We use the following standard facts.

Fact (Limit rule). *Let $f, g > 0$ and suppose $L = \lim_{n \rightarrow \infty} f(n)/g(n)$ exists in $[0, \infty]$. If $0 < L < \infty$ then $f = \Theta(g)$; if $L = 0$ then $f = O(g)$ but $f \neq \Omega(g)$; if $L = \infty$ then $f = \Omega(g)$ but $f \neq O(g)$.*

Fact (Logs vs. polynomials). *For any constants $a, b > 0$, $\lim_{n \rightarrow \infty} (\log n)^b / n^a = 0$. (Substituting $n = 2^t$, this is $t^b / 2^{at} \rightarrow 0$: an exponential beats any polynomial.)*

	$f(n)$	$g(n)$	Answer
(a)	$n - 100$	$n - 200$	$f = \Theta(g)$
(b)	$n^{1/2}$	$n^{2/3}$	$f = O(g)$
(c)	$100n + \log n$	$n + (\log n)^2$	$f = \Theta(g)$
(d)	$n \log n$	$10n \log 10n$	$f = \Theta(g)$
(e)	$\log 2n$	$\log 3n$	$f = \Theta(g)$
(f)	$10 \log n$	$\log(n^2)$	$f = \Theta(g)$
(g)	$n^{1.01}$	$n \log^2 n$	$f = \Omega(g)$
(h)	$n^2 / \log n$	$n(\log n)^2$	$f = \Omega(g)$
(i)	$n^{0.1}$	$(\log n)^{10}$	$f = \Omega(g)$
(j)	$(\log n)^{\log n}$	$n / \log n$	$f = \Omega(g)$
(k)	$\sqrt{n}$	$(\log n)^3$	$f = \Omega(g)$
(l)	$n^{1/2}$	$5^{\log_2 n}$	$f = O(g)$
(m)	$n2^n$	3^n	$f = O(g)$
(n)	2^n	2^{n+1}	$f = \Theta(g)$
(o)	$n!$	2^n	$f = \Omega(g)$
(p)	$(\log n)^{\log n}$	$2^{(\log_2 n)^2}$	$f = O(g)$
(q)	$\sum_{i=1}^n i^k$	n^{k+1}	$f = \Theta(g)$

Wherever the answer is only O or only Ω , the other relation fails (the ratio tends to 0 or ∞).

Justifications.

- (a) $\frac{n-100}{n-200} \rightarrow 1$.
- (b) $\frac{n^{1/2}}{n^{2/3}} = n^{-1/6} \rightarrow 0$.
- (c) $\frac{100n + \log n}{n + (\log n)^2} = \frac{100 + (\log n)/n}{1 + (\log n)^2/n} \rightarrow 100$.
- (d) $\frac{n \log n}{10n \log(10n)} = \frac{\log n}{10(\log n + \log 10)} \rightarrow \frac{1}{10}$.
- (e) $\frac{\log 2n}{\log 3n} = \frac{1 + \log n}{\log 3 + \log n} \rightarrow 1$.
- (f) $\log(n^2) = 2 \log n$, so $f/g = 5$ exactly.
- (g) $\frac{n^{1.01}}{n \log^2 n} = \frac{n^{0.01}}{(\log n)^2} \rightarrow \infty$.
- (h) $\frac{n^2 / \log n}{n(\log n)^2} = \frac{n}{(\log n)^3} \rightarrow \infty$.
- (i) $\frac{n^{0.1}}{(\log n)^{10}} \rightarrow \infty$.
- (j) $(\log n)^{\log n} = 2^{\log n \cdot \log \log n} = n^{\log \log n}$. For $n \geq 16$, $\log \log n \geq 2$, so $\frac{f}{g} = n^{\log \log n - 1} \log n \geq n \rightarrow \infty$.
- (k) $\frac{\sqrt{n}}{(\log n)^3} \rightarrow \infty$.
- (l) $5^{\log_2 n} = 2^{\log_2 5 \cdot \log_2 n} = n^{\log_2 5}$ with $\log_2 5 \approx 2.32$, so $f/g = n^{1/2 - \log_2 5} \rightarrow 0$.
- (m) $\frac{n2^n}{3^n} = n \left(\frac{2}{3}\right)^n \rightarrow 0$.
- (n) $2^{n+1} = 2 \cdot 2^n$, so $f/g = \frac{1}{2}$.
- (o) $\frac{n!}{2^n} = \prod_{i=1}^n \frac{i}{2} \geq \frac{1}{2} \left(\frac{3}{2}\right)^{n-2} \rightarrow \infty$ for $n \geq 2$ (the first factor is $\frac{1}{2}$, the second is 1, and every later factor is at least $\frac{3}{2}$).
- (p) As in (j), $f = 2^{\log n \cdot \log \log n}$, while $g = 2^{(\log n)^2}$. Hence $f/g = 2^{-\log n (\log n - \log \log n)} \rightarrow 0$, since $\log n - \log \log n \rightarrow \infty$.
- (q) Here $k \geq 0$ is a fixed constant. Upper bound: each of the n terms is at most n^k , so $\sum_{i=1}^n i^k \leq n^{k+1}$. Lower bound: the terms with $i \geq \lceil n/2 \rceil$ number at least $n/2$ and each is at least $(n/2)^k$, so $\sum_{i=1}^n i^k \geq (n/2)^{k+1} = 2^{-(k+1)} n^{k+1}$. Thus $f = \Theta(n^{k+1})$.

Remark. Changing the base of a logarithm multiplies it by a constant, so none of the answers depend on the base (in (j) and (p) the logarithm is in the exponent, but the same comparisons go through with $\ln$).

Exercise 0.2

Let $g(n) = \sum_{i=0}^n c^i$ with $c > 0$. All terms are positive.

(a) $c < 1$. Since $g(n)$ contains the term 1 and all terms are positive, $g(n) \geq 1$. Also

$$g(n) \leq \sum_{i=0}^{\infty} c^i = \frac{1}{1-c},$$

a constant independent of n . Hence $1 \leq g(n) \leq \frac{1}{1-c}$ for all n , i.e. $g(n) = \Theta(1)$.

(b) $c = 1$. Every term equals 1, so $g(n) = n + 1$. For $n \geq 1$, $n \leq n + 1 \leq 2n$, hence $g(n) = \Theta(n)$.

(c) $c > 1$. The last term gives $g(n) \geq c^n$. By the geometric sum formula,

$$g(n) = \frac{c^{n+1} - 1}{c - 1} < \frac{c^{n+1}}{c - 1} = \frac{c}{c - 1} \cdot c^n.$$

Since $\frac{c}{c-1}$ is a positive constant, $c^n \leq g(n) \leq \frac{c}{c-1} c^n$, i.e. $g(n) = \Theta(c^n)$.

Exercise 1.14

Idea. Combine the matrix method of Exercise 0.4 with modular repeated squaring. Let $X = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$. Since $\begin{pmatrix} F_{k+1} \\ F_{k+2} \end{pmatrix} = X \begin{pmatrix} F_k \\ F_{k+1} \end{pmatrix}$, induction gives

$$\begin{pmatrix} F_n \\ F_{n+1} \end{pmatrix} = X^n \begin{pmatrix} F_0 \\ F_1 \end{pmatrix} = X^n \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

so F_n is the top-right entry $(X^n)_{12}$. It therefore suffices to compute $X^n \bmod p$ (every entry reduced modulo p).

Algorithm 1 Computing $F_n \bmod p$

```

1: function MATPOW( $n, p$ ) ▷ returns  $X^n \bmod p$ 
2:   if  $n = 0$  then
3:     return  $I_2 \bmod p$ 
4:   end if
5:    $Z \leftarrow \text{MATPOW}(\lfloor n/2 \rfloor, p)$ 
6:    $Z \leftarrow Z \cdot Z \bmod p$ 
7:   if  $n$  is odd then
8:      $Z \leftarrow X \cdot Z \bmod p$ 
9:   end if
10:  return  $Z$ 
11: end function
12: function FIBMOD( $n, p$ )
13:  return the top-right entry of MATPOW( $n, p$ )
14: end function

```

Here “ $A \cdot B \bmod p$ ” means the 2×2 matrix product (8 multiplications and 4 additions of entries, Exercise 0.4(a)) followed by reducing each of the 4 entries modulo p .

Correctness. MATPOW follows the recursion $X^n = (X^{\lfloor n/2 \rfloor})^2$ for even n and $X^n = X \cdot (X^{\lfloor n/2 \rfloor})^2$ for odd n , exactly as in modular exponentiation. Every matrix entry is computed from others using only $+$ and $\times$, and reduction modulo p commutes with these operations ($a \equiv a', b \equiv b' \Rightarrow a + b \equiv a' + b', ab \equiv a'b' \pmod{p}$). Hence by induction on n , each entry of the returned matrix is congruent modulo p to the corresponding entry of X^n , and lies in $\{0, \dots, p-1\}$. In particular FIBMOD returns $F_n \bmod p$.

Running time. Let $k = \lceil \log(p+1) \rceil$ be the number of bits of p .

- The recursion depth is $\lfloor \log n \rfloor + 1 = O(\log n)$, and each level performs at most two matrix multiplications, so there are $O(\log n)$ matrix multiplications in total.
- All entries are in $[0, p)$, i.e. at most k bits. One matrix multiplication does 8 products of k -bit numbers ($O(k^2)$ each), 4 additions of $2k$ -bit numbers ($O(k)$ each), and 4 reductions of $O(k)$ -bit numbers modulo p ($O(k^2)$ each). So one matrix multiplication costs $O(k^2)$.

The total running time is

$$O(\log n \cdot \log^2 p)$$

bit operations, which is polynomial in the input size ($\approx \log n + \log p$ bits). By contrast, running `fib2` and reducing modulo p at every step takes n additions, i.e. $O(n \log p)$ time, which is exponential in the length $\log n$ of n .

Exercise 1.20

We use the extended Euclidean algorithm: a has an inverse modulo N iff $\gcd(a, N) = 1$, and if $ax + Ny = 1$ then $x \bmod N$ is the inverse.

1. $20^{-1} \bmod 79$.

$$79 = 3 \cdot 20 + 19, \quad 20 = 1 \cdot 19 + 1.$$

Back-substituting: $1 = 20 - 19 = 20 - (79 - 3 \cdot 20) = 4 \cdot 20 - 1 \cdot 79$. Hence $20^{-1} \equiv \boxed{4} \pmod{79}$.
Check: $20 \cdot 4 = 80 = 79 + 1$.

2. $3^{-1} \bmod 62$.

$$62 = 20 \cdot 3 + 2, \quad 3 = 1 \cdot 2 + 1.$$

Back-substituting: $1 = 3 - 2 = 3 - (62 - 20 \cdot 3) = 21 \cdot 3 - 1 \cdot 62$. Hence $3^{-1} \equiv \boxed{21} \pmod{62}$.
Check: $3 \cdot 21 = 63 = 62 + 1$.

3. $21^{-1} \bmod 91$.

$$91 = 4 \cdot 21 + 7, \quad 21 = 3 \cdot 7 + 0,$$

so $\gcd(21, 91) = 7 \neq 1$ (indeed $21 = 3 \cdot 7$, $91 = 13 \cdot 7$). Therefore **21 has no inverse modulo 91**: if $21x \equiv 1 \pmod{91}$, then 7 would divide $21x - 91y = 1$, which is impossible.

4. $5^{-1} \bmod 23$.

$$23 = 4 \cdot 5 + 3, \quad 5 = 1 \cdot 3 + 2, \quad 3 = 1 \cdot 2 + 1.$$

Back-substituting:

$$1 = 3 - 2 = 3 - (5 - 3) = 2 \cdot 3 - 5 = 2(23 - 4 \cdot 5) - 5 = 2 \cdot 23 - 9 \cdot 5.$$

Hence $5^{-1} \equiv -9 \equiv \boxed{14} \pmod{23}$. Check: $5 \cdot 14 = 70 = 3 \cdot 23 + 1$.

Exercise 1.31

Let N be an n -bit number, so $2^{n-1} \leq N < 2^n$ and $n - 1 \leq \log N < n$.

(a) The number of bits of $N!$ is $\lfloor \log N! \rfloor + 1 = \Theta(\log N!)$. We show $\log N! = \Theta(N \log N)$ (Exercise 1.4).

- Upper bound: $N! \leq N^N$, so $\log N! \leq N \log N$.
- Lower bound: at least $N/2$ of the factors $1, \dots, N$ are $\geq N/2$, so $N! \geq (N/2)^{N/2}$ and $\log N! \geq \frac{N}{2}(\log N - 1) \geq \frac{N}{4} \log N$ for $N \geq 4$.

Since $N = \Theta(2^n)$ and $\log N = \Theta(n)$, the length of $N!$ is

$$\Theta(N \log N) = \boxed{\Theta(n 2^n)} \text{ bits.}$$

(b) Algorithm: multiply the numbers one by one.

Algorithm 2 Computing $N!$

```

1: function FACTORIAL( $N$ )
2:    $f \leftarrow 1$ 
3:   for  $i \leftarrow 2$  to  $N$  do
4:      $f \leftarrow f \cdot i$  ▷ invariant: after this line,  $f = i!$ 
5:   end for
6:   return  $f$ 
7: end function

```

Correctness is immediate from the loop invariant $f = i!$.

Running time. Multiplying a b -bit number by an m -bit number with $m \leq b$ by the grade-school method adds up m shifted copies of the b -bit number, each addition costing $O(b + m) = O(b)$; so it takes $O(bm)$ time.

In iteration i , the current value $f = (i - 1)! \leq N!$ has $b = O(\log N!) = O(N \log N) = O(Nn)$ bits by part (a), and $i \leq N$ has at most n bits. Hence each iteration costs $O(Nn \cdot n) = O(Nn^2)$ (the loop counter update is only $O(n)$). With $N - 1$ iterations, the total time is

$$O(N \cdot Nn^2) = O(N^2 n^2) = \boxed{O(n^2 4^n)}.$$

This bound is tight for this algorithm: in each of the last $N/2$ iterations ($N/2 < i \leq N$), $f = (i - 1)! \geq (N/2)!$ already has $\Omega(N \log N) = \Omega(Nn)$ bits, and i has at least $n - 1$ bits, so these iterations alone cost $\Omega(N \cdot Nn \cdot n)$ under the grade-school method.

Remark. The running time is exponential in the input length n . This is unavoidable for *any* algorithm: by (a), merely writing down the output takes $\Theta(n2^n)$ time.

Exercise 1.35

(a) x is its own inverse modulo p iff $x^2 \equiv 1 \pmod{p}$, i.e. $p \mid x^2 - 1 = (x - 1)(x + 1)$. Since p is prime, this holds iff $p \mid x - 1$ or $p \mid x + 1$, i.e. $x \equiv 1$ or $x \equiv -1 \pmod{p}$. For $1 \leq x < p$, the self-inverse numbers are therefore exactly

$$x = 1 \quad \text{and} \quad x = p - 1$$

(for $p = 2$ these coincide, leaving only $x = 1$).

(b) If $p = 2$: $(p - 1)! = 1 \equiv -1 \pmod{2}$. Now let p be an odd prime and $S = \{2, 3, \dots, p - 2\}$.

Every $x \in S$ has a unique inverse $x^{-1} \in \{1, \dots, p - 1\}$ modulo p . We claim $x^{-1} \in S$ and $x^{-1} \neq x$:

- If $x^{-1} = 1$, then $x = 1$; if $x^{-1} = p - 1$, then $x = (p - 1)^{-1} = p - 1$ (as $p - 1$ is self-inverse). Both contradict $x \in S$, so $x^{-1} \in S$.
- $x^{-1} \neq x$ by part (a), since $x \notin \{1, p - 1\}$.

Moreover $(x^{-1})^{-1} = x$. Hence S splits into disjoint pairs $\{x, x^{-1}\}$, each with product $\equiv 1 \pmod{p}$, and so $\prod_{x \in S} x \equiv 1 \pmod{p}$ (for $p = 3$, $S = \emptyset$ and the empty product is 1). Therefore

$$(p - 1)! = 1 \cdot \left(\prod_{x \in S} x \right) \cdot (p - 1) \equiv 1 \cdot 1 \cdot (-1) = -1 \pmod{p}.$$

(c) Wilson's theorem concerns $N \geq 2$ (modulo 1 all integers are congruent, so $N = 1$ is excluded). Let $N \geq 2$ be composite and write $N = ab$ with $1 < a < N$. Since $a \leq N - 1$, a is one of the factors of $(N - 1)!$, so $a \mid (N - 1)!$. Also $a \mid N$. Hence

$$d = \gcd(N, (N - 1)!) \geq a > 1.$$

Suppose, for contradiction, that $(N - 1)! \equiv -1 \pmod{N}$, i.e. $(N - 1)! = kN - 1$ for some integer k . Then $d \mid N$ and $d \mid (N - 1)!$ imply $d \mid kN - (N - 1)! = 1$, so $d = 1$, a contradiction. Thus $(N - 1)! \not\equiv -1 \pmod{N}$.

(d) To use the test we must compute $(N - 1)! \bmod N$. For an n -bit N the straightforward way performs $N - 2 = \Theta(2^n)$ modular multiplications (each $O(n^2)$), which is exponential in the input size n . Unlike $a^{N-1} \bmod N$ in Fermat's test, which repeated squaring computes with only $O(n)$ multiplications because all factors are equal, the factors $1, 2, \dots, N - 1$ of $(N - 1)!$ are all different, and no polynomial-time algorithm for computing $(N - 1)! \bmod N$ is known. So Wilson's theorem, although an exact characterization of primes, does not yield an efficient primality test.